

File Handling and Stream I/O in C

> **Definition to Remember**

> File
device

1. Concept Overview & Fundamentals

File Handling and Stream I/O in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Co

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization

```text

+-----

### Core Architecture Breakdown

| Component / Concept | Technical Specification | Operational Characteristics | Performance / Complexity |

| :--- | :

---

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
```c
// Core standard implementation pattern

// Core logic execution

return status;
```

#include
int mai
printf("
}
```

---

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

- **E**

---

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: append log

**Problem Description & Objective:**

**\*\*C Code Implementation:\*\***

```
int main() {
```

```
FILE *fp = fopen("log.txt", "a");
```

```
if (fp == NULL) {
```

```
printf("Enter text to append: ");
```

```
fputs(text, fp);
```

```
fclose(fp);
```

```

```

### **Problem 2: check file open**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
int main() {
```

```
char filename[100];
```

```
printf("Enter filename: ");
```

```
scanf("%s", filename);
```

```
FILE *fp = fopen(filename, "r");
```

```
if (fp == NULL) {
```

```
printf("File not found.\n");
```

```
return 0;
```

```
}
```

```

```

### Problem 3: copy file

**\*\*Problem Description & Objective:\*\***

Implement a C program to copy the contents of one file to another.

**\*\*C Code Implementation:\*\***

```
```c
```

```
int main() {
```

```
/*
```

```
FILE *src = fopen("source.txt", "r");
```

```
FILE *dest = fopen("dest.txt", "w");
```

```
char ch;
```

```
if (src == NULL || dest == NULL) {
```

```
printf("Error: File not found.\n");
```

```
while ((ch = fgetc(src)) != EOF) {
```

```
fputc(ch, dest);
```

```
printf("File copied successfully!\n");
```

```
fclose(src);
```

```
fclose(dest);
```

```
return 0;
```

```
}
```

Problem 4: read write file

****Problem Description & Objective:****

Implemen

****C Code Implementation:****

```c

int main() {

/\*

FILE \*fp = fopen("data.txt", "w+");

char te

if (fp == NULL) {

printf("

// writing

printf("

// move pointer to start

rewind

// reading

printf("

fclose(fp);

return

---

#### Problem 5: sum from file

**\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C Code Implementation:\*\***

```
int main() {
```

```
FILE *fp = fopen("numbers.txt", "r");
```

```
if (fp == NULL) {
```

```
while (fscanf(fp, "%d", &num) != EOF) {
```

```
printf("Sum = %d\n", sum);
```

```
fclose(fp);
```

```

```

### **Problem 6: write lines**

**\*\*Problem Description & Objective:\*\***

**\*\*C Code Implementation:\*\***

```
int main() {
```

```
FILE *fp = fopen("output.txt", "w");
```

```
if (fp == NULL) {
```

```
printf("Error: File not found.\n");
```

```
printf("How many lines? ");
```

```
scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {
```

```
printf("Line %d: ", i + 1);
```

```
fclose(fp);
```

```
return 0;
```

```

```

## 6. Exam Scoring Points & Memory Revision

> **Must-Write Points**

> 1. File

> **Quick Recall**

> `fopen`